~~How To Brick Your PC~~

Reverse Engineering Windows Kernel Mode Drivers

SteelCon 2025 🎉

Michael Rowley

michaellrowley@protonmail.com / https://www.linkedin.com/in/michaellrowley/

Michael Rowley

Day job as a software engineer

Break stuff in my spare time

- Gained a few CVEs
- Published a few write-ups
- Made a few tools
- Exploit research/mentor for CSIRT.global
- Member of CVE's Automation Working Group

Usual Disclaimer

Anything that I say as part of this workshop is entirely unrelated to my employment or association with any entity/group. Thoughts and opinions are my own, and actions are taken as an individual without any endorsement from or towards myself. No criticism is intended towards any individual, group, or organisation.

Also, If you break something, don't blame me :)

This workshop is **TLP:CLEAR**

michaellrowley@protonmail.com / https://www.linkedin.com/in/michaellrowley/

Why Are We Here?

- Windows kernel-mode drivers are cool (when they aren't busy crashing)
- Taking them apart is a great way to learn about foundational concepts
- Vulnerabilities in kernel-mode drivers can have significant implications



What Will You Gain?

- A basic understanding of how the Windows kernel works
- Ability to spot common mistakes made by driver developers
- Ability to reverse engineer drivers and understand their functionality

Who Is This For?

Everyone!

But…

A basic understanding of C or assembly or intermediate representation (IR) would be great

Today’s Agenda:

= hands-on
= presented

Introduction

- What is the Windows kernel? + why should I care?
- What do Windows kernel-mode drivers do? TL;DR: everything!
- How do drivers work?

Understanding Capability

- Symbols & Imports
- Finding The Entrypoint
- Finding Comms Mechanisms IRPs, callbacks, device objects
- Understanding IRPs for Comms
- Dynamic Analysis WinDBG, emulation

Triaging Vulnerabilities

- MSI Afterburner: RTCore64
- AMD: amdfendrsys
- HTTP Debugger: NetFilterSDK
- What about kernel security mechanisms? If we have time!

Preparation!

Assets are split across 3 folders

Or https://gist.github.com/michaellrowley/953d4c565967ecdc713b38877131ae87

https://www.michaelrowley.dev/STEELCON/RESOURCES.txt

1 Toolset & Script Links



Ghidra



Binary Ninja Cloud



PE Imports

2 Driver Samples



AMD Crash Defender



HTTP Debugger SDK



RTCore64

3 Other Links

ReactOS

Windows Research Kernel

Cheat Sheet

MSDN Area for KMDF

Bonus Links

Preparation!

Assets are split across 3 folders

https://www.michaelrowley.dev/STEELCON

1 Toolset & Script Links



Ghidra



Binary Ninja Cloud



PE Imports

Select Your Favourite

If you didn't install Ghidra in advance, sign up for Binary Ninja Cloud

2 Driver Samples



AMD Crash Defender



HTTP Debugger SDK



RTCore64

3 Other Links

ReactOS

Windows Research Kernel

Cheat Sheet

MSDN Area for KMDF

Bonus Links

Preparation!

Assets are split across 3 folders

https://www.michaelrowley.dev/STEELCON

1 Toolset & Script Links


Ghidra


Binary Ninja Cloud


PE Imports

2 Driver Samples


AMD Crash Defender


HTTP Debugger SDK


RTCore64

3 Other Links

ReactOS

Windows Research Kernel

Cheat Sheet

MSDN Area for KMDF

Bonus Links

These are either patched or vulnerable

Preparation!

Assets are split across 3 folders

https://www.michaelrowley.dev/STEELCON

1 Toolset & Script Links



Ghidra



Binary Ninja Cloud



PE Imports

2 Driver Samples



AMD Crash Defender



HTTP Debugger SDK



RTCore64

3 Other Links

ReactOS

Windows Research Kernel

Cheat Sheet

MSDN Area for KMDF

Public Codebases

Very similar to the current Windows kernel

Preparation!

Assets are split across 3 folders

https://www.michaelrowley.dev/STEELCON

1 Toolset & Script Links



Ghidra



Binary Ninja Cloud



PE Imports

2 Driver Samples



AMD Crash Defender



HTTP Debugger SDK



RTCore64

3 Other Links

ReactOS

Windows Research Kernel

Cheat Sheet

MSDN Area for KMDF

Your Workshop Guide

Please use it!

Preparation!

Assets are split across 3 folders

https://www.michaelrowley.dev/STEELCON

1 Toolset & Script Links



Ghidra



Binary Ninja Cloud



PE Imports

2 Driver Samples



AMD Crash Defender



HTTP Debugger SDK



RTCore64

3 Other Links

ReactOS

Windows Research Kernel

Cheat Sheet

MSDN Area for KMDF

Bonus Links

Official MS Documentation

For the Windows Kernel Mode Driver Framework

Introduction

Levels of System Privilege



User-Mode

Device Drivers

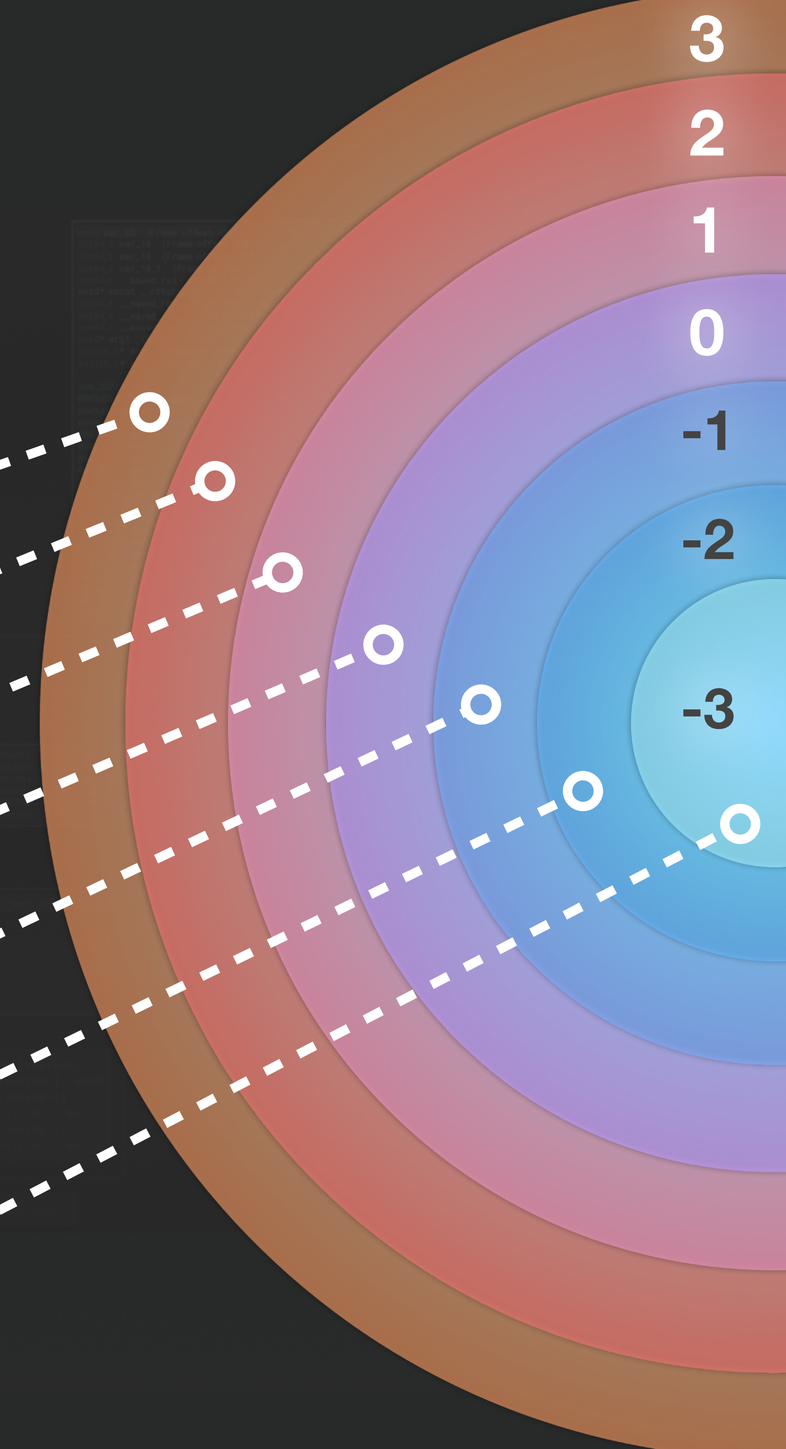
(Also) Device Drivers

Kernel

Hypervisor

System Management Mode

Management Engine

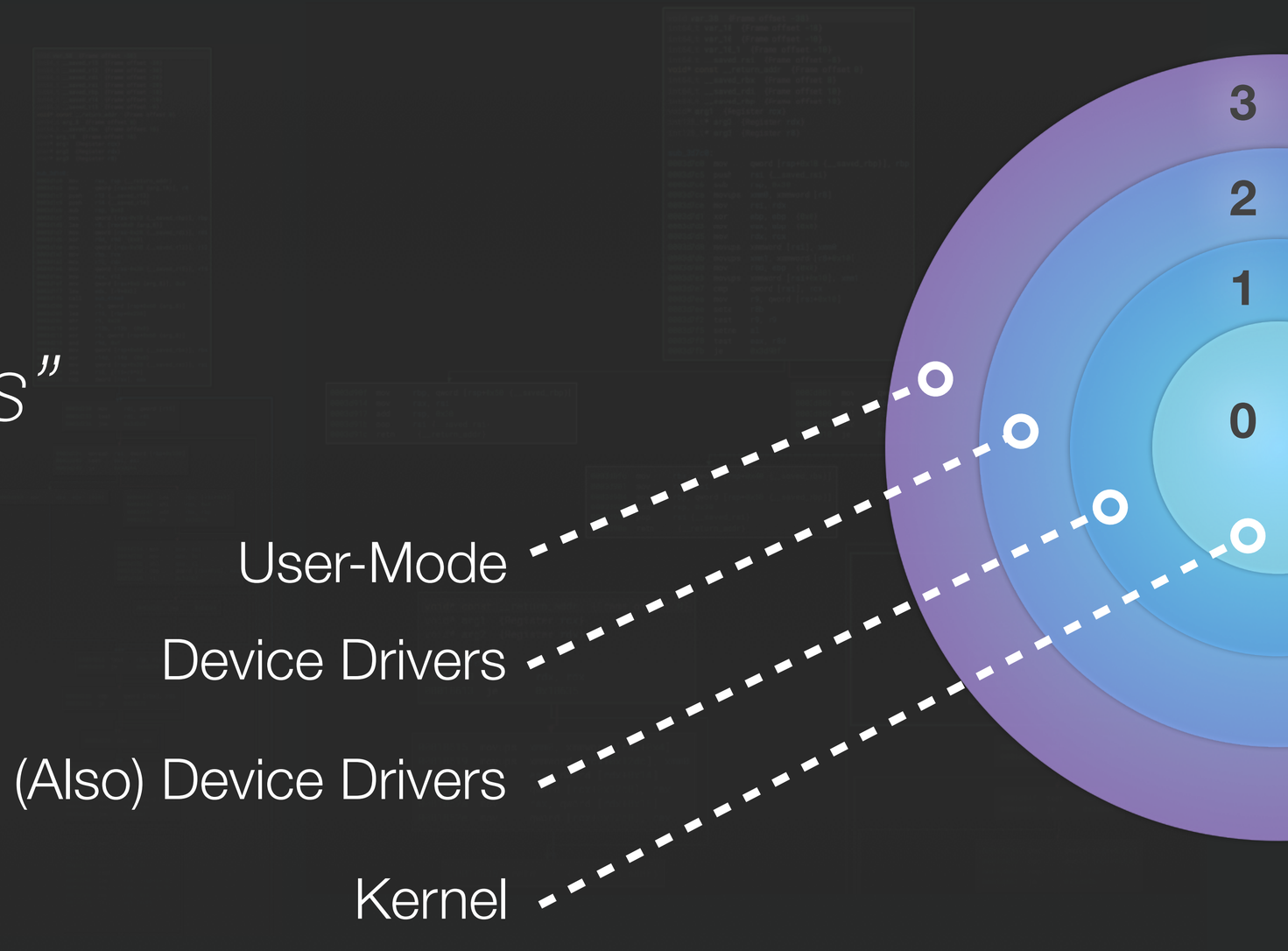


Introduction

Levels of Operating System Privilege

“Operating Systems”

(In theory)



Introduction

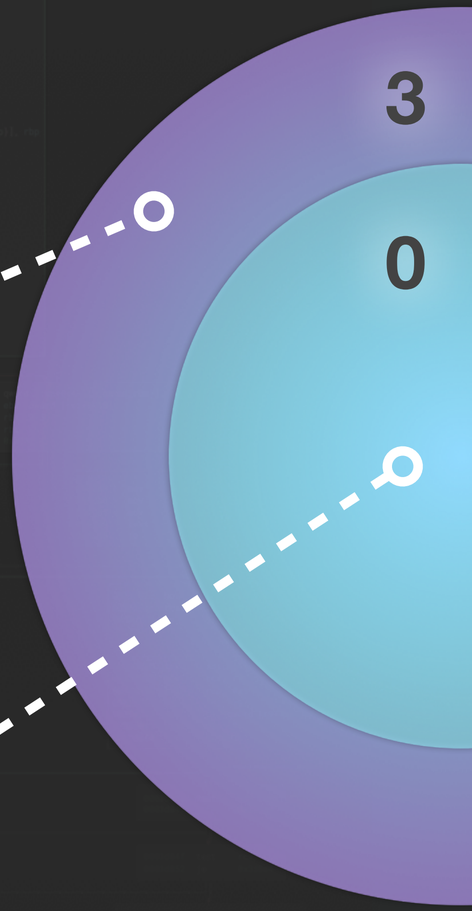
Levels of *Implemented* Operating System Privilege



User-Mode

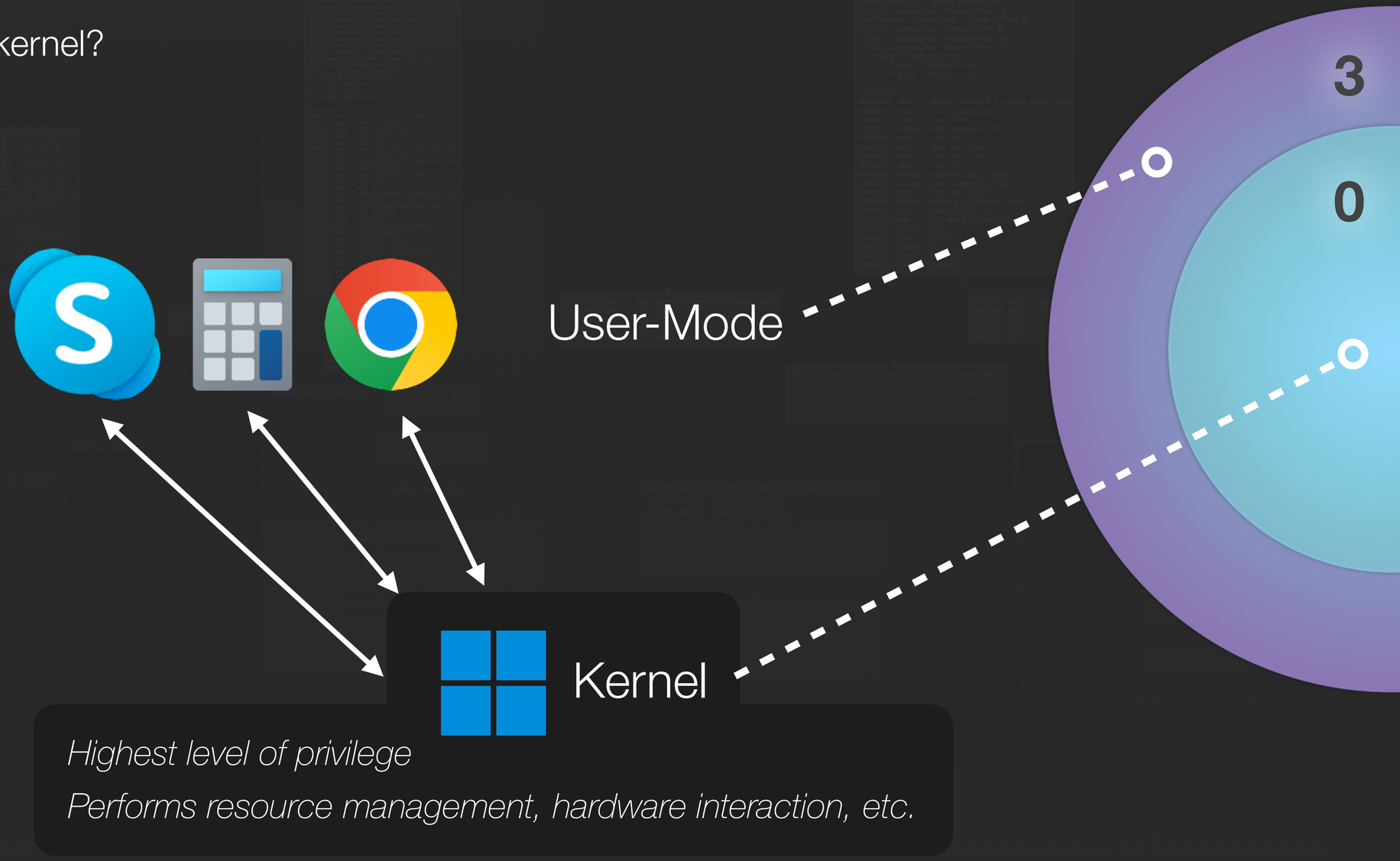


Device Drivers + Kernel



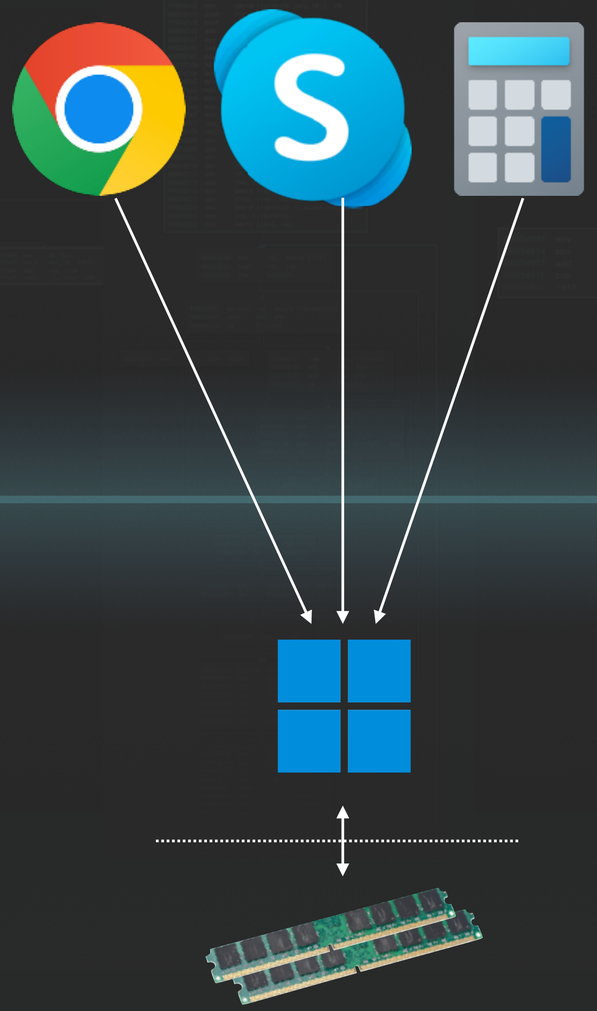
Introduction

What is the Windows kernel?



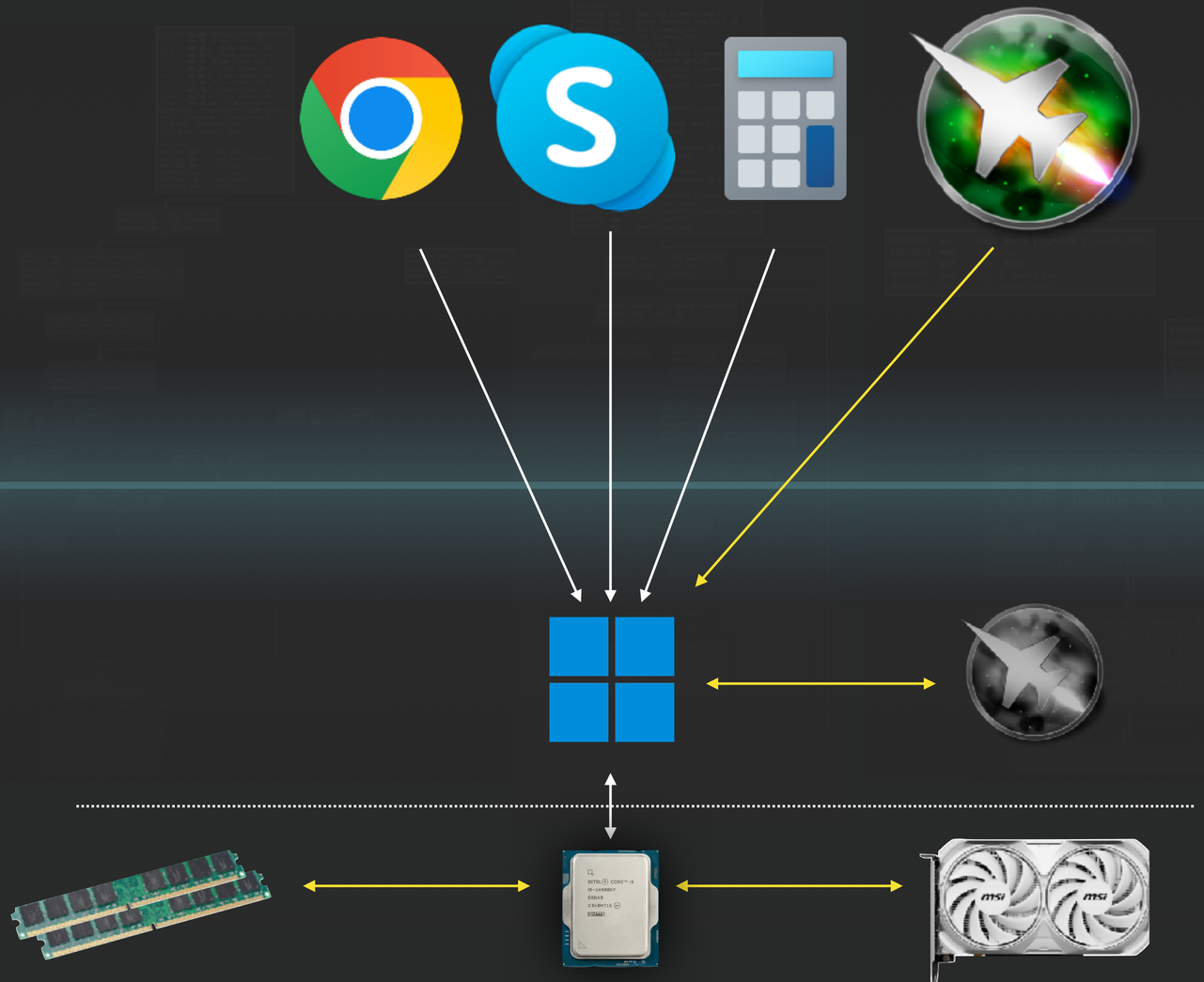
Introduction

What is a kernel-mode driver?



Introduction

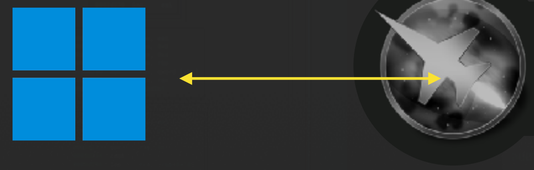
What is a kernel-mode driver?



Introduction

What is a kernel-mode driver?

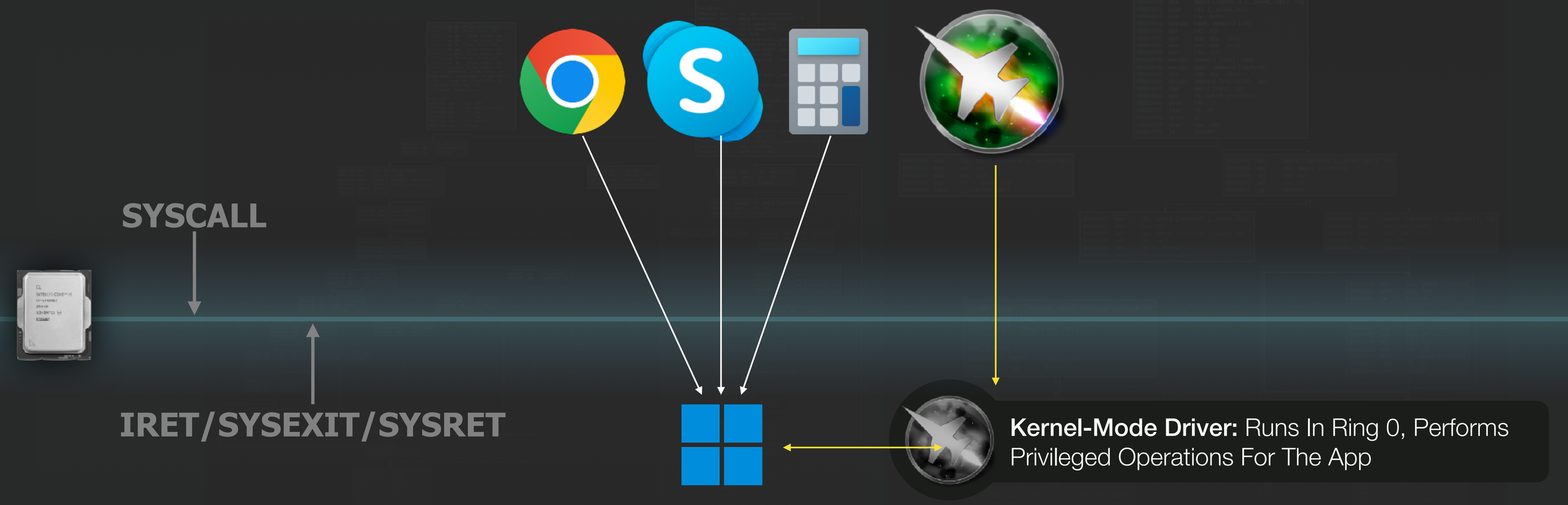




Kernel-Mode Driver: Runs In Ring 0, Performs Privileged Operations For The App

Introduction

What is a kernel-mode driver?



Introduction

How do kernel-mode drivers work?



Introduction

How do kernel-mode drivers work?





Mostly
(+ ASM)

Some

Others in small quantities

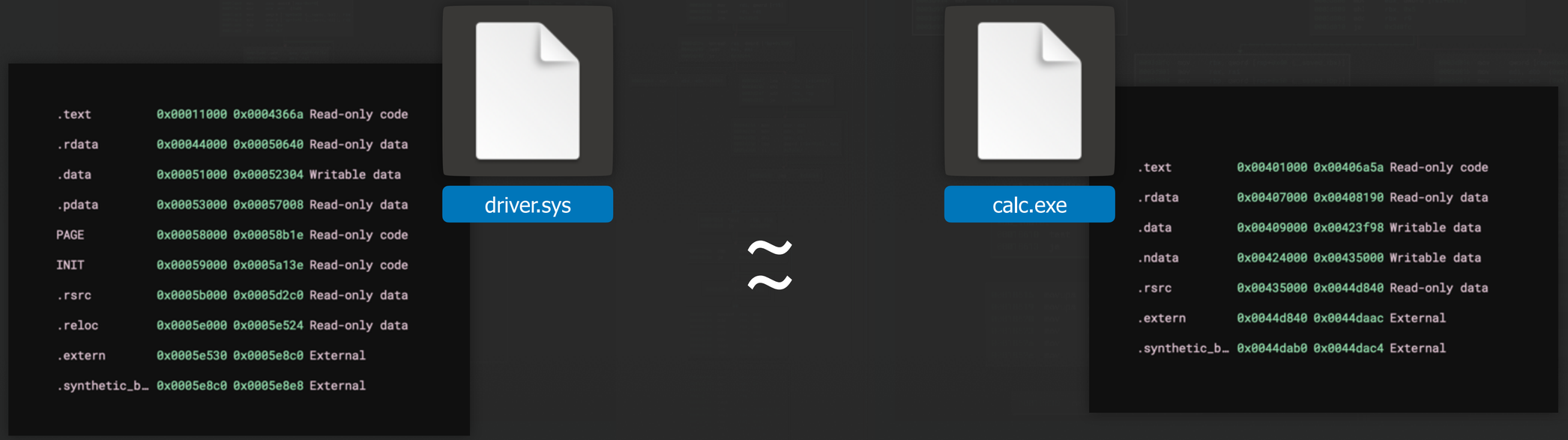
Increasing

Introduction

How do kernel-mode drivers work?

Same PE32+ Format as User-Mode Binaries

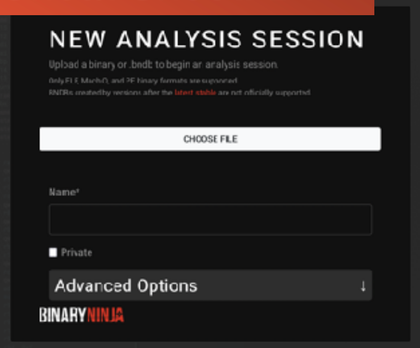
(Mostly)

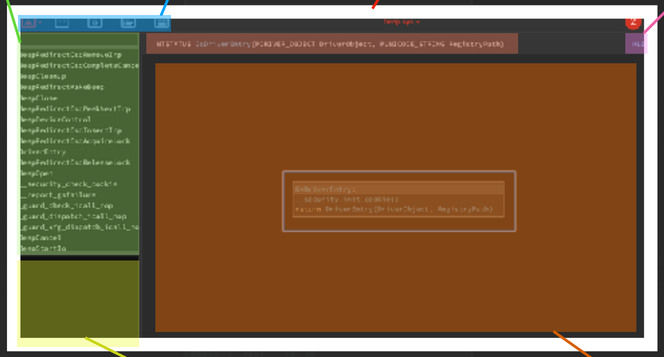


Reverse Engineering 101

Start your ~~engines~~ tools!

Cheat Sheet: B1



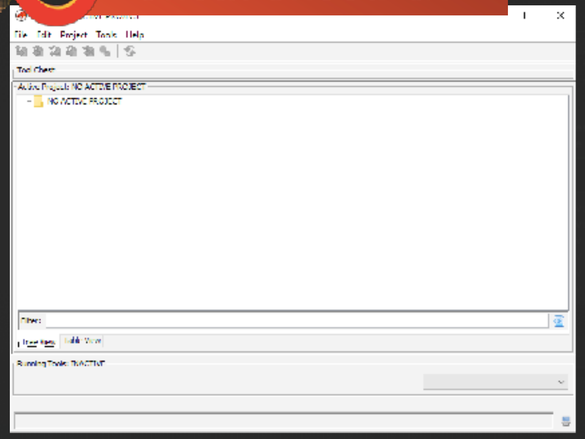


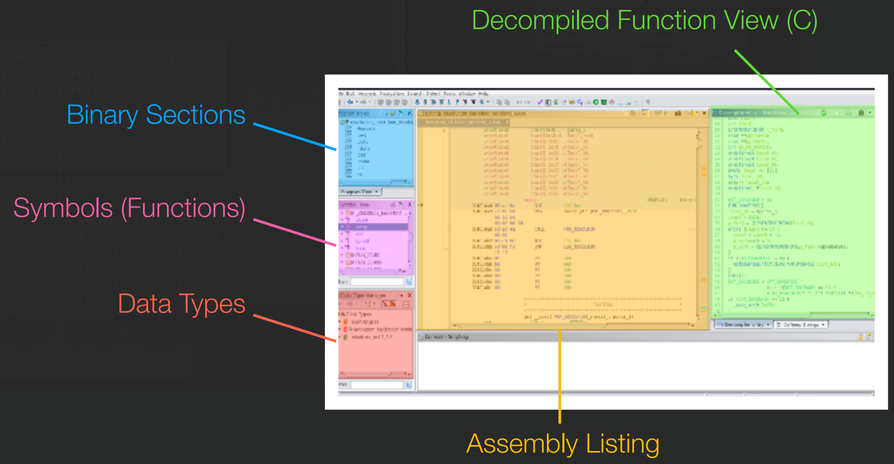
1. Open your reverse engineering tool



2. Load a sample for analysis

Cheat Sheet: G1





3. Explore the User Interface!

Reverse Engineering 101

Symbols & Imports

Note: Imports and symbols aren't always available

Symbols are invaluable for learning reverse engineering functionality and structures

Imports allow you to understand which parts of the Windows kernel (and other dependencies) are used by your sample



PE Imports

Option: Mass enumerate imports from all drivers on a system to identify 'interesting' potential targets.

```
A:/Windows/System32/sioctl.sys
        ntoskrnl.exe!RtlInitUnicodeString/null @ 0x00402000
        ntoskrnl.exe!ProbeForRead/null @ 0x00402004
        ntoskrnl.exe!MmProbeAndLockPages/null @ 0x00402008
        ntoskrnl.exe!MmUnlockPages/null @ 0x0040200C
        ntoskrnl.exe!MmMapLockedPagesSpecifyCache/null @ 0x00402010
        ntoskrnl.exe!IoAllocateMdl/null @ 0x00402014
        ntoskrnl.exe!IofCompleteRequest/null @ 0x00402018
        ntoskrnl.exe!IoCreateDevice/null @ 0x0040201C
        ntoskrnl.exe!IoCreateSymbolicLink/null @ 0x00402020
        ntoskrnl.exe!IoDeleteDevice/null @ 0x00402024
        ntoskrnl.exe!IoDeleteSymbolicLink/null @ 0x00402028
        ntoskrnl.exe!IoFreeMdl/null @ 0x0040202C
        ntoskrnl.exe!memcpy/null @ 0x00402030
        ntoskrnl.exe!RtlUnwind/null @ 0x00402034
B:/Windows/System32/netio.sys
        ntoskrnl.exe!ExRegisterCallback/null @ 0x0005E000
```

Understanding Capabilities

How do kernel-mode drivers work? Driver Basics

'Gs' security wrapper around DriverEntry when /GS is passed at compile-time.
The wrapper provides "stack security level checking".

KMDF drivers always have their entrypoint as (Gs)DriverEntry

```
NTSTATUS GsDriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
```

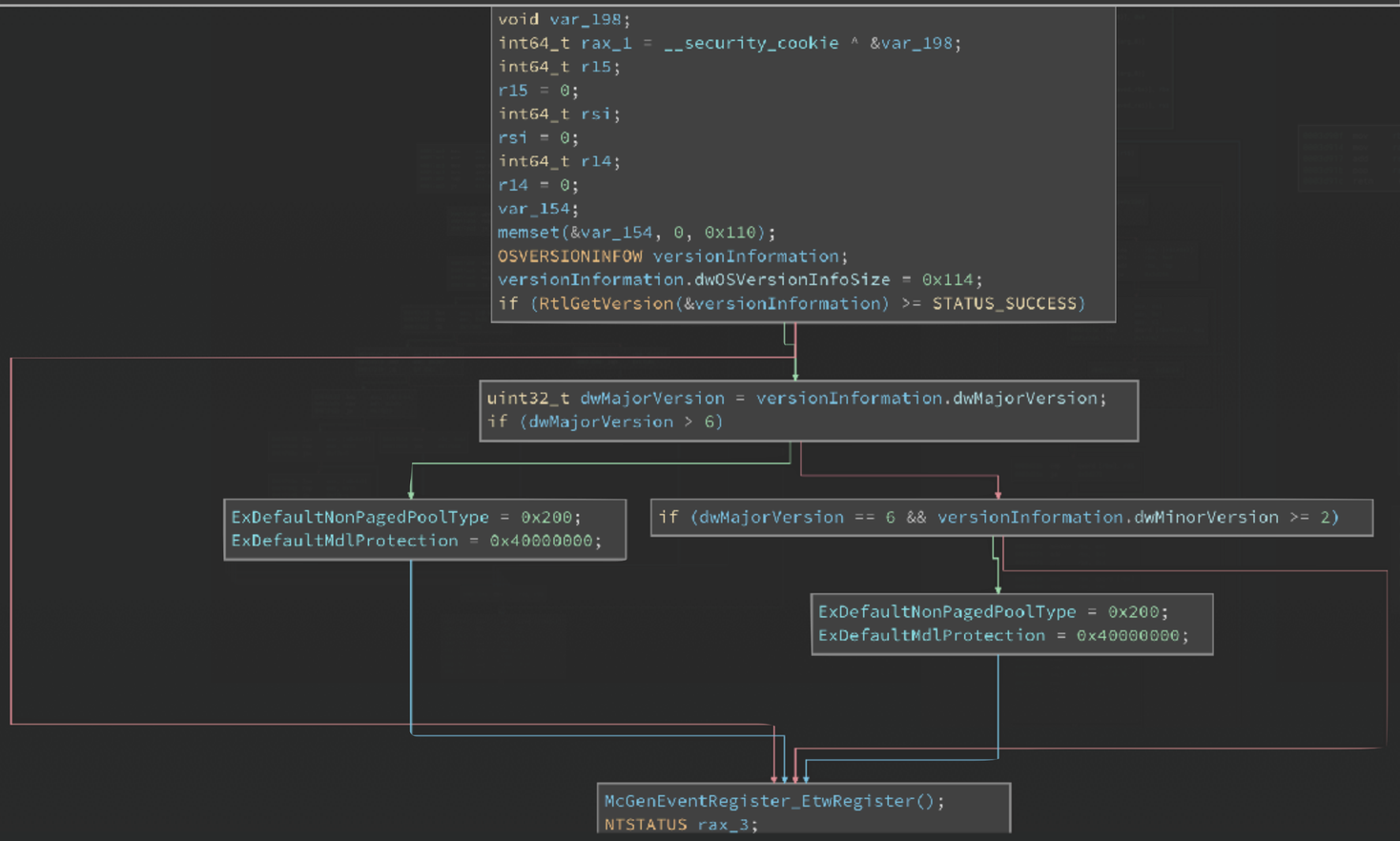
```
GsDriverEntry:
__security_init_cookie()
return DriverEntry(DriverObject, RegistryPath)
```

Core initialisation/functionality

Kernel Mode Driver Framework

Driver Entrypoint

```
uint64_t DriverEntry(DRIVER_OBJECT* arg1, int16_t* arg2)
```



1. Locate the driver's entrypoint
2. 'Annotate' the disassembly
3. What functions do you see?

Learn / Windows / Windows Drivers

Cheat Sheet: B2

Cheat Sheet: G2

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS DriverEntry(DRIVER_OBJECT* driver, PUNICODE_STRING regKey);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework



Cheat Sheet: B3



Cheat Sheet: G3

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object

FILE_DEVICE_SECURE_OPEN



2. Register “Interrupt Service Routines” (ISRs) in the device

3. Create a symbolic link for user-mode creation of device handles

4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateDevice(
  PDRIVER_OBJECT DriverObject, ULONG DeviceExtensionSize,
  PUNICODE_STRING DeviceName, DEVICE_TYPE DeviceType,
  ULONG DeviceCharacteristics, BOOLEAN Exclusive, PDEVICE_OBJECT* DeviceObject
);
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

Kernel Mode Driver Framework

Common Parts of a DriverEntry



Cheat Sheet: B4



Cheat Sheet: G4

```
DriverObject->MajorFunction[IRP_MJ_XXX] = HandlerFn
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
DriverObject->MajorFunction[IRP_MJ_XXX] = HandlerFn
```

1. Create a device object
2. **Register “Interrupt Service Routines” (ISRs) in the device**
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/irp-major-function-codes

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
IRP_MJ_WRITE
IRP_MJ_READ
IRP_MJ_CREATE
DriverObject->MajorFunction[IRP_MJ_XXX] = HandlerFn
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nc-wdm-driver_dispatch

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
DriverObject->MajorFunction[IRP_MJ_XXX] = HandlerFn
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatesymboliclink

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateSymbolicLink(
  PUNICODE_STRING LinkName, PUNICODE_STRING DeviceName
)
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. **Create a symbolic link for user-mode creation of device handles**
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatesymboliclink

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateSymbolicLink(
  PUNICODE_STRING LinkName, PUNICODE_STRING DeviceName
)
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. **Create a symbolic link for user-mode creation of device handles**
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatesymboliclink

Kernel Mode Driver Framework

Common Parts of a DriverEntry

```
NTSTATUS IoCreateSymbolicLink(
  PUNICODE_STRING LinkName, PUNICODE_STRING DeviceName
)
```

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. **Create a symbolic link for user-mode creation of device handles**
4. Do other initialisation, register other callbacks, etc.

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatesymboliclink

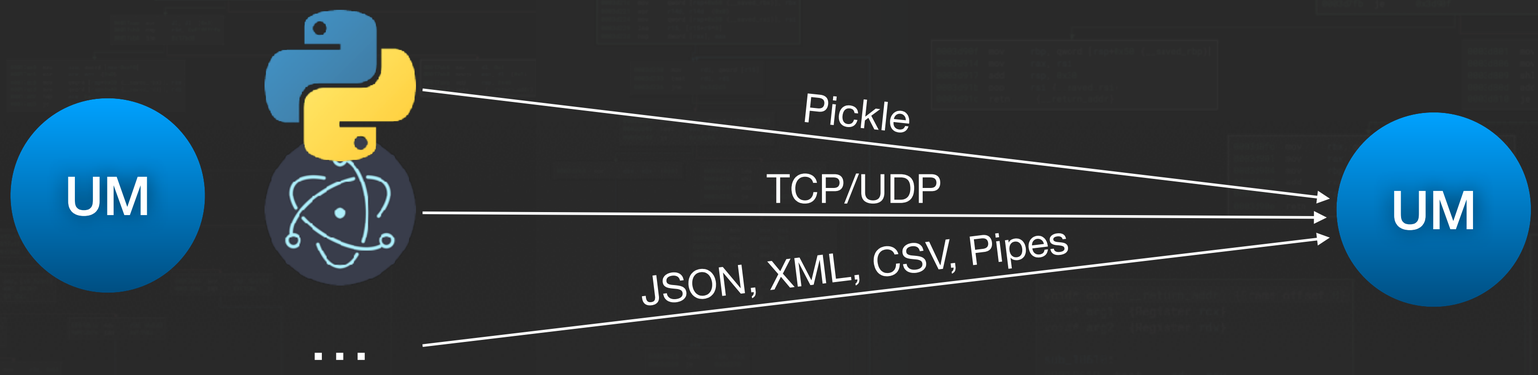
Kernel Mode Driver Framework

Common Parts of a DriverEntry

1. Create a device object
2. Register “Interrupt Service Routines” (ISRs) in the device
3. Create a symbolic link for user-mode creation of device handles
4. **Do other initialisation, register other callbacks, etc.**

Kernel/User-Mode Communication

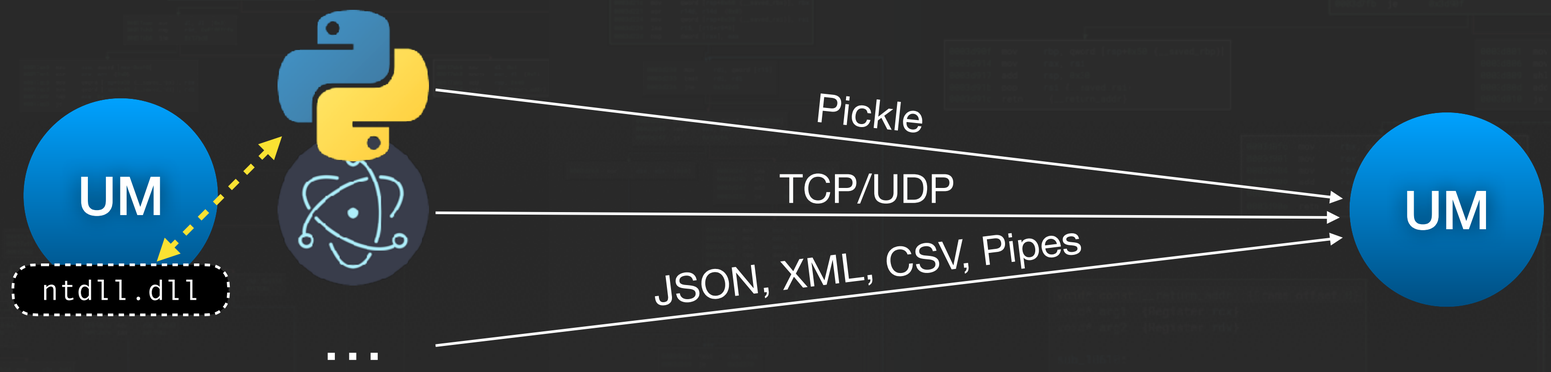
UM User-Mode Application



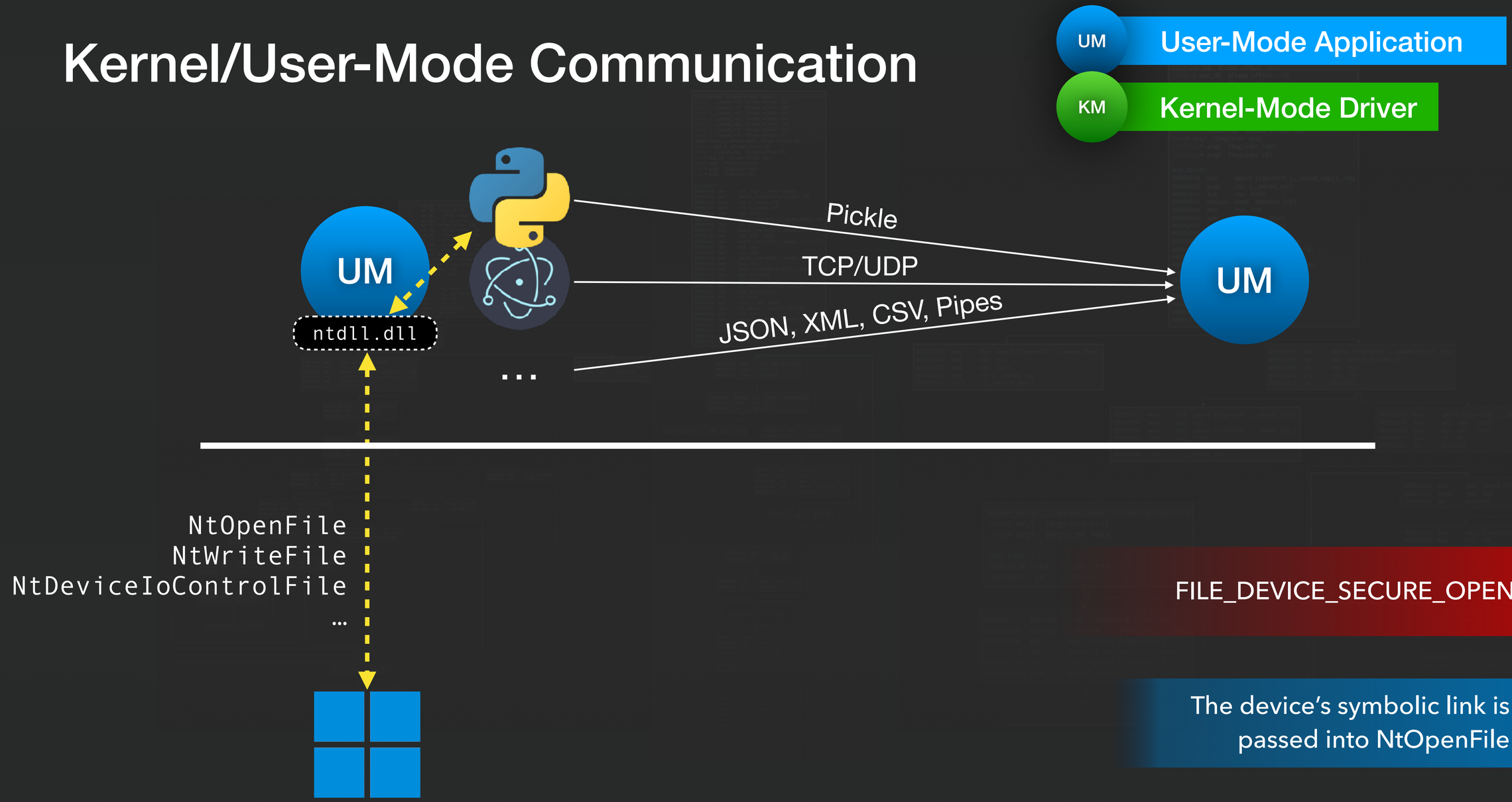
Kernel/User-Mode Communication

UM User-Mode Application

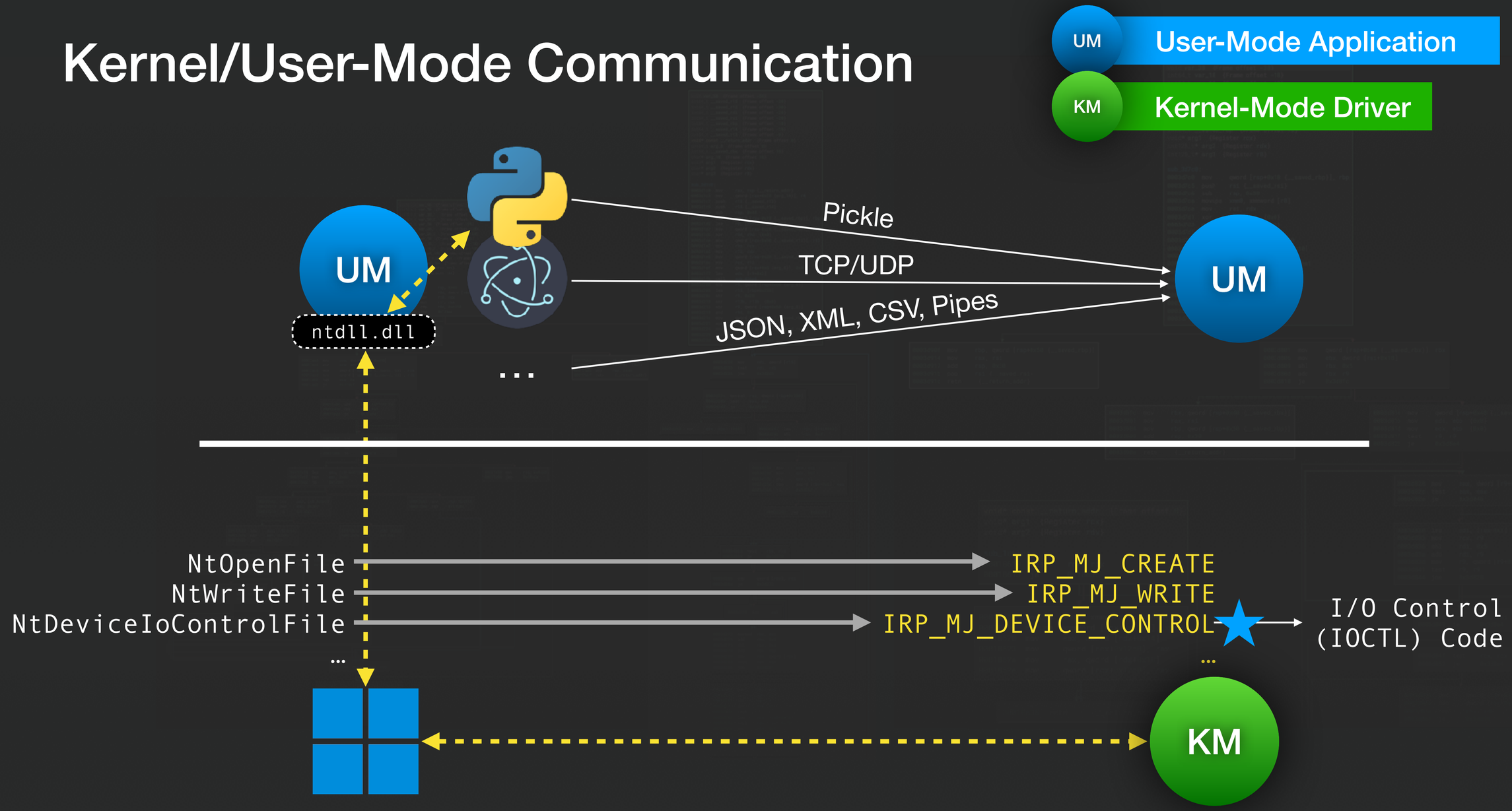
NTDLL: usually called via Kernel32.dll !



Kernel/User-Mode Communication



Kernel/User-Mode Communication



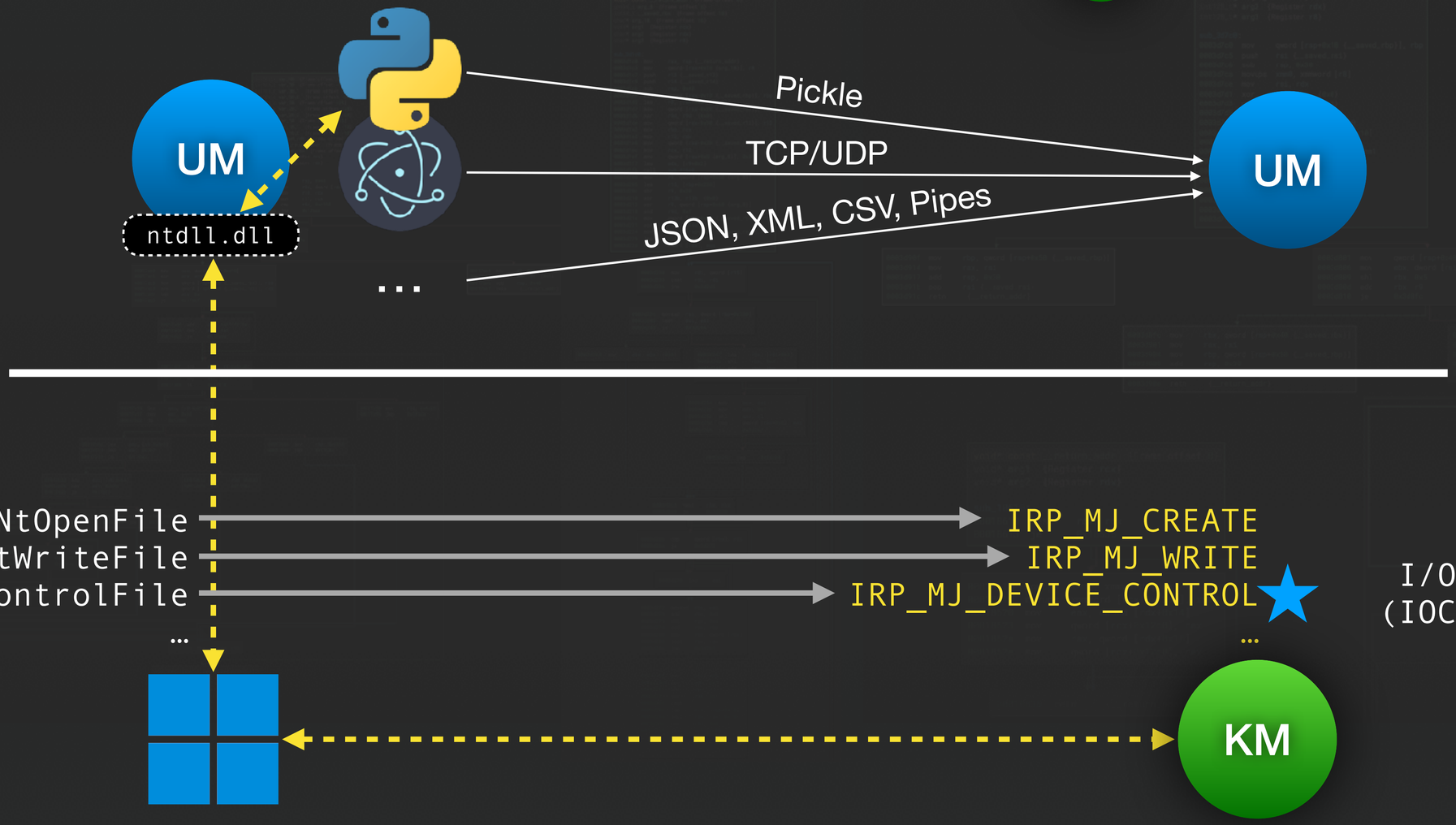
Cheat Sheet: B5

Cheat Sheet: G5

Kernel/User-Mode Communication

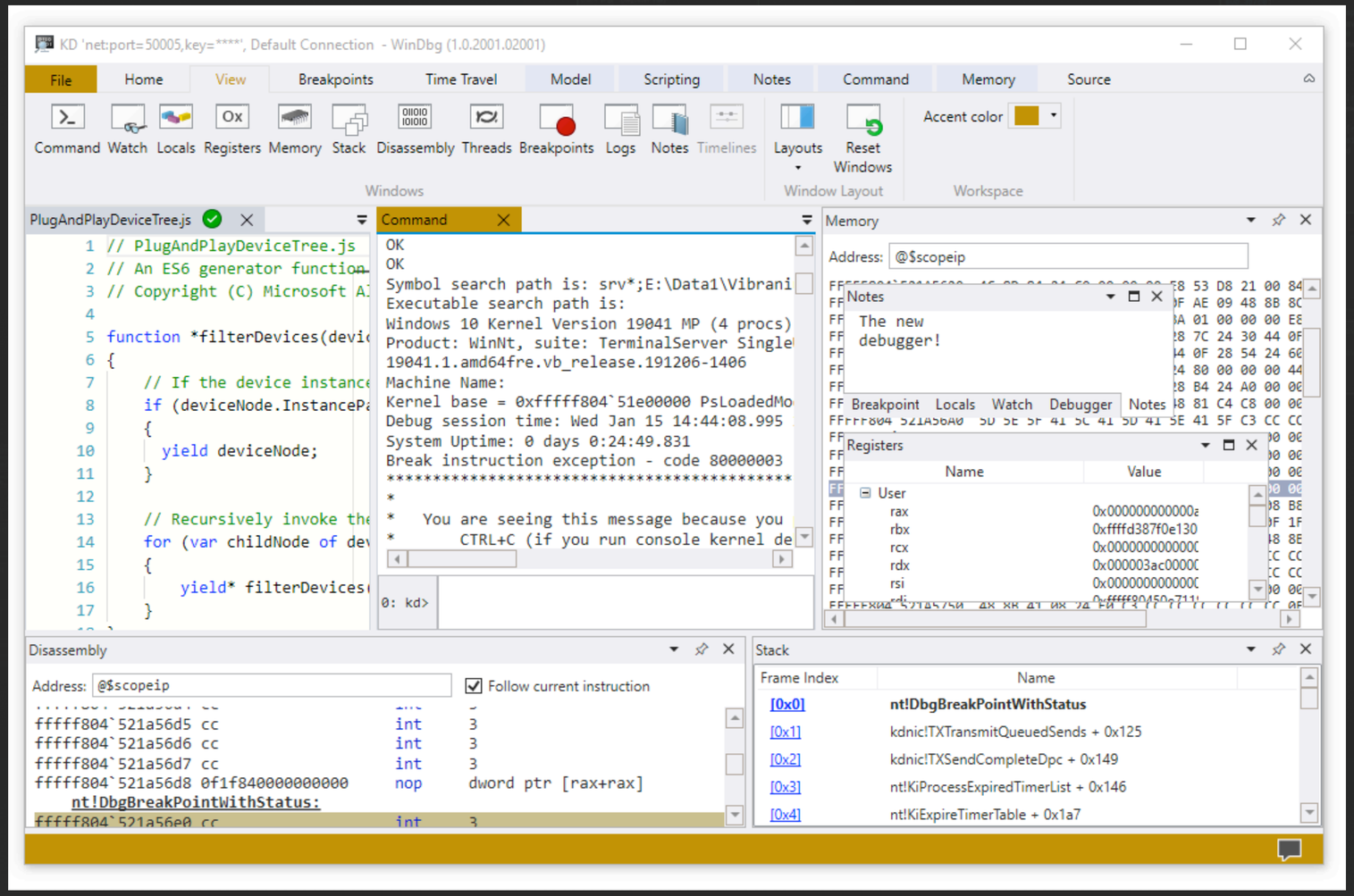
UM User-Mode Application

KM Kernel-Mode Driver



Bonus: Dynamic Analysis

WinDbg: An official Microsoft tool



Dynamic Analysis

X86 Emulation with Windows kernel-compatible stubs

These are possible examples, **not recommendations or endorsements!**



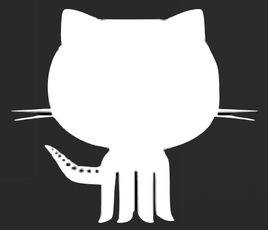
Strongly recommended that you make/ modify a tool bespoke for your use-case





Bochs





binsnake/KUBERA



x64dbg

Triaging Some Vulnerabilities!



AMD
Crash Defender Driver



HTTP Debugger
And a few other drivers



MSI Afterburner
RTCore Driver